MusicTUI: A Terminal-Based Music Player with Synced Lyrics, ANSI Half-Block Colour Art, and Event-Driven Architecture

MusicTUI Project

github.com/RosarioLC/musictui

October 24, 2026

ABSTRACT

This paper presents MusicTUI, a terminal-based music player built with Python 3.12+ and the Textual framework. The application streams music from YouTube Music, manages playlists and liked songs, displays synchronized lyrics, and renders album artwork as ANSI half-block colour text within a responsive terminal user interface. The system employs a clean layered architecture comprising presentation, service, domain, and storage layers, with Protocol-based dependency injection enabling extensibility and testability. An event-driven communication model using a central EventBus decouples the playback engine from UI widgets, eliminating polling and reducing coupling. Domain models such as Track, PlayState, and LyricLine are implemented as immutable frozen dataclasses with slot optimization for memory efficiency. The application leverages python-vlc for audio playback, SQLite for persistent storage with schema migration, PIL and NumPy for image downscaling to ANSI half-block colour art, and a triple-cached LRU cache for tracks, lyrics, and stream URLs. Synchronized lyrics are fetched via a multi-tier fallback system (ytmusicapi → lrclib → syncedlyrics → linear distribution). The paper documents the complete architecture, design decisions, implementation details, testing methodology, and future work for this open-source project.

Keywords terminal user interface, music player, Python, Textual framework, synchronous lyrics, ANSI art, event-driven architecture, dependency injection, SQLite, VLC media library

1 Introduction

Terminal-based applications have experienced a renaissance in recent years, driven by developer familiarity with command-line environments, the ubiquity of SSH-based remote workflows, and the growing capability of modern terminal emulators to render rich colour graphics. Within this landscape, music players have traditionally occupied a niche dominated by minimal, text-oriented tools such as cmus, ncmpcpp, and mpc. While these tools are powerful, they often lack the visual polish and interactive features that users expect from modern media consumption software.

The MusicTUI project addresses this gap by delivering a feature-rich music player that operates entirely within the terminal yet provides a visual experience comparable to

desktop applications. Built on the `Textual` framework [1], which provides a reactive `TUI` component model with CSS-like styling, `MusicTUI` renders album artwork as full-colour `ANSI` half-block characters, displays synchronized lyrics with auto-highlighting, and offers a complete playlist management system – all within a responsive, keyboard-driven interface.

The primary contributions of this project are:

1. A `Protocol`-based layered architecture that cleanly separates concerns across presentation, service, domain, and storage layers.
2. An event-driven communication model using a central `EventBus` that decouples the playback engine from UI components.
3. A novel `ANSI` half-block colour image rendering pipeline using `PIL` and `NumPy` that converts album artwork into terminal-compatible colour art.
4. A multi-tier lyrics fetching system with three fallback layers ensuring maximum coverage for synced lyrics.
5. An immutable domain model using `Python`'s frozen dataclasses with slot optimization for thread safety and memory efficiency.
6. A comprehensive test suite covering all layers of the application with in-memory mocks and `SQLite` databases.

The remainder of this paper is organized as follows. Section 2 describes the system architecture and design philosophy. Section 3 details the domain models. Section 4 covers the playback subsystem. Section 5 discusses the catalog integration with YouTube Music. Section 6 presents the UI architecture. Section 7 documents the storage layer. Section 8 describes the testing strategy. Section 9 covers installation and usage. Section 10 outlines future work. Section 11 concludes.

2 System Architecture

`MusicTUI` employs a four-layer architecture that enforces unidirectional data flow and dependency inversion. Figure 1 illustrates the overall layered structure.

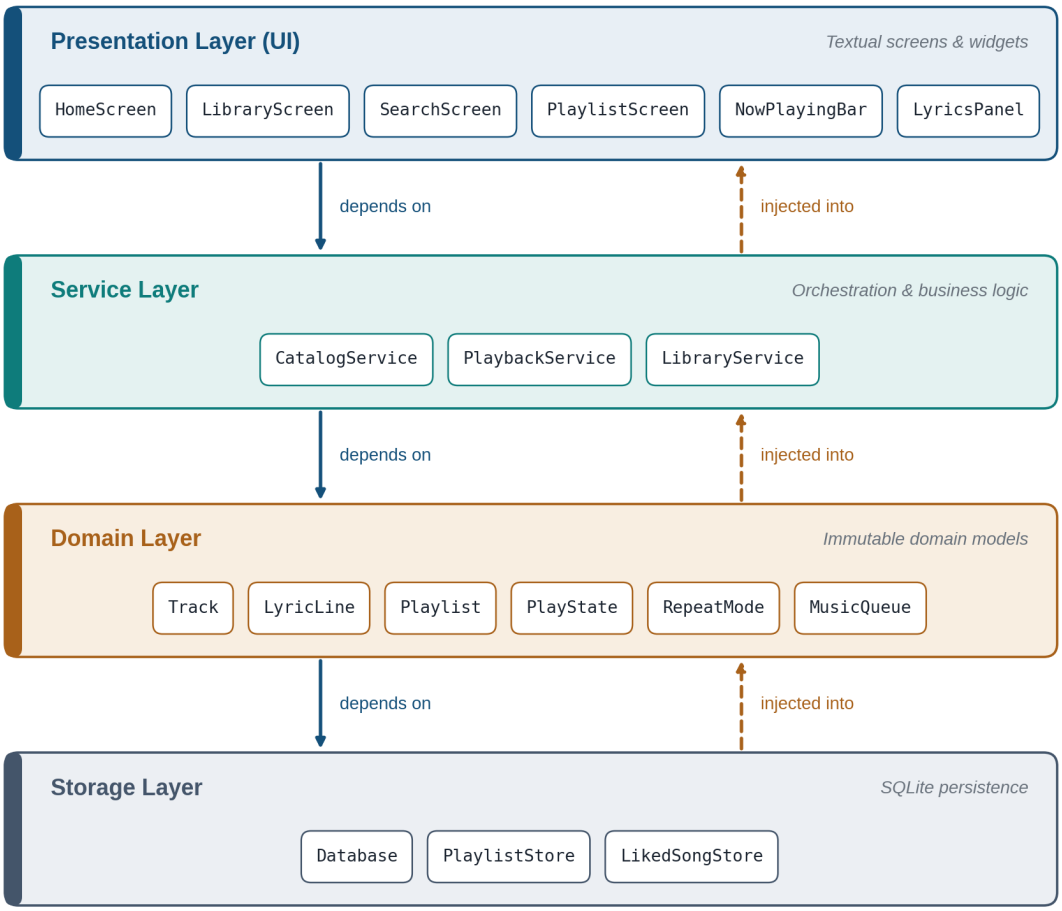


Figure 1. Layered architecture overview showing the four layers of `MusicTUI` and their interdependencies. Arrows indicate that higher layers depend on lower layers, with dependency injection flowing in the opposite direction.

2.1 Layer Descriptions

Presentation Layer (UI) The presentation layer comprises the `Textual` application class `MusicTUI`(App), six screen classes (`HomeScreen`, `SearchScreen`, `LibraryScreen`, `PlaylistScreen`, `FullscreenScreen`, `LyricsScreen`), and twenty-two reusable widget classes. This layer is responsible solely for rendering state and capturing user input. It contains no business logic; all interaction with application state is delegated to the service layer through the `EventBus`.

The UI follows a grid-based layout with a two-column, three-row structure: the left column contains a collapsible sidebar for navigation, while the right column holds the main content area. The bottom row is occupied by the persistent `NowPlayingBar`, which is

composited across all screens. CSS-like styling is defined in `style.tcss`, a 597-line theme file that implements a dark Spotify-inspired palette with #121212 background and #1ed760 green accent colour.

Service Layer The service layer consists of three service classes that aggregate domain objects and coordinate cross-cutting concerns:

- `CatalogService` wraps a `CatalogSource` implementation and a `TrackCache`, providing search, track retrieval, lyrics fetching, and stream URL resolution with automatic caching.
- `PlaybackService` wraps the `Player` and `EventBus`, exposing an event-driven API for playback control that screens and widgets use exclusively.
- `LibraryService` wraps the `PlaylistStore`, providing playlist CRUD operations, liked-songs management, and song addition/removal.

Domain Layer The domain layer contains the core business models: `Track`, `LyricLine`, `PlayState`, `Playlist`, and `RepeatMode`. These models are immutable (frozen dataclasses) and implement value-based equality and hashing. The domain layer has no dependencies on external libraries or frameworks.

Storage Layer The storage layer provides persistent data management through `SQLite`: `PlaylistStore` handles playlist and song persistence with track resolution, while `LikedSongStore` manages the liked-songs collection. The `Database` class provides a context-manager-wrapped SQLite connection with automatic schema creation and migration support.

2.2 Dependency Injection

`MusicTUI` uses `Protocol`-based dependency injection, a form of structural typing where dependencies are specified as `Python` `Protocol` classes rather than concrete implementations. Two key protocols define this pattern:

Design Decision

CatalogSource Protocol: Any class implementing `search(query)`, `get_track(id)`, `get_track_stream_url(track)`, and `get_lyrics(track)` can serve as a music catalog provider. The current implementation is `YTMusicSource`, but the protocol enables future providers (local file system, Spotify, etc.) to be injected without modifying consuming code.

Design Decision

AudioBackend Protocol: Any class implementing `play(url)`, `pause()`, `resume()`, `stop()`, `seek(position_ms)`, `get_position()`, `get_duration()`, `set_volume(volume)`, `get_volume()`, and properties `is_playing` and `is_paused` can serve as the audio backend.

The current implementation is `VLCAudioBackend`, but the protocol enables testing with `FakeBackend` and future replacement with alternative backends.

The wiring diagram is as follows:

```
MusicTUI (App)
  ├── YTMusicSource ──┐
  ├── TrackCache      ├──> CatalogService
  │                   ┘
  ├── VLCAudioBackend ──┐
  ├── MusicQueue        ├──> Player ──> PlaybackService
  │                     ┘
  ├── Database ──> PlaylistStore ──> LibraryService
  └── EventBus ──┘
```

3 Domain Models

The domain models in `MusicTUI` are implemented as immutable frozen dataclasses using `Python` 3.10+'s `@dataclass(slots=True, frozen=True)` decorator. This design choice provides three key benefits:

1. **Thread safety:** Frozen objects cannot be mutated after construction, eliminating race conditions in the multi-threaded `Textual` worker model.
2. **Memory efficiency:** The `slots=True` parameter eliminates the `__dict__` per-instance dictionary, reducing memory overhead by approximately 40% compared to standard dataclasses.
3. **Functional updates:** The `replace(**changes)` pattern produces new instances with modified fields, following the immutable update idiom common in functional programming.

3.1 Track Model

The `Track` dataclass is the central data structure of the application, representing a single music track with full metadata:

Listing 1. The `Track` frozen dataclass with default value normalization and functional update support.

```python
@dataclass(slots=True, frozen=True)
class Track:
    id: int
    title: str = "Unknown Title"
    artist: str = "Unknown Artist"
    album: str = "Unknown Album"
    duration_ms: int = 0
    image_path: str = "Unknown Image Path"
    lyrics_path: str = "Unknown Lyrics Path"
    url: str = ""
    provider_id: str = ""
```

```
    def __post_init__(self) -> None:
        if not self.title or not self.title.strip():
            object.__setattr__(self, "title", "Unknown Title")
        if self.duration_ms < 0:
            object.__setattr__(self, "duration_ms", 0)

    def to_dict(self) -> dict[str, object]:
        return {field: getattr(self, field)
                for field in self.__dataclass_fields__}

    def __hash__(self) -> int: return hash(self.id)
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Track): return NotImplemented
        return self.id == other.id

    def replace(self, **changes: object) -> Track:
        return replace(self, **changes)
```

Key design decisions:

- Track IDs are computed as `zlib.crc32(video_id) & 0xFFFFFFFF`, producing deterministic 32-bit unsigned integers from YouTube video IDs.
- The `to_dict()` method enables straightforward serialization for database storage.
- Equality and hashing are based solely on `id`, allowing tracks to be used in sets and as dictionary keys.

3.2 LyricLine Model

The `LyricLine` dataclass represents a single line of synchronized lyrics:

Listing 2. The `LyricLine` frozen dataclass.

```
@dataclass(slots=True, frozen=True)
class LyricLine:
    timestamp_ms: int
    text: str
```

3.3 PlayState Model

The `PlayState` dataclass captures a snapshot of playback state:

Listing 3. The `PlayState` frozen dataclass.

```
@dataclass(slots=True, frozen=True)
class PlayState:
    is_playing: bool = False
    is_paused: bool = False

    def replace(self, **changes: object) -> PlayState:
        return replace(self, **changes)
```

The dual boolean fields allow all four states: not playing/playing, paused/not paused. In practice, `is_playing=True` and `is_paused=True` together represent the paused state.

3.4 Playlist Model

The `Playlist` class is a mutable ordered collection that maintains both a list and a set of track IDs for O(1) membership testing:

Architecture Note

Dual-structure invariant: The `Playlist` class maintains `songs` (a `list[Track]` for ordered iteration and index-based access) and `song_ids` (a `set[int]` for O(1) membership checks). All mutations (`add_song`, `remove_song`) update both structures atomically, preserving the invariant that `t.id in song_ids` if and only if `t in songs`.

The cursor-based navigation (`next_song`, `previous_song`) supports sequential track traversal within a playlist context.

3.5 RepeatMode Enum

The `RepeatMode` enumeration defines three repeat modes using modular arithmetic:

Listing 4. RepeatMode enum with cycle logic.

```
class RepeatMode(Enum):
    OFF = 0
    ALL = 1
    ONE = 2

    def next_mode(self) -> RepeatMode:
        return RepeatMode((self.value + 1) % 3)
```

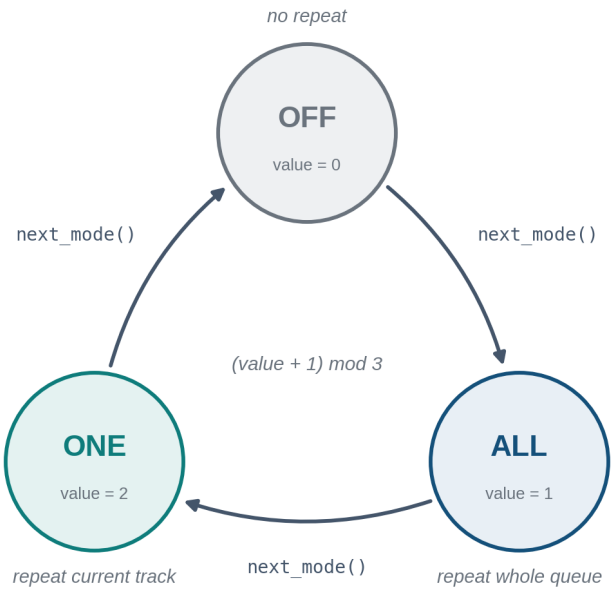


Figure 2. The repeat mode cycle diagram showing the three states and transitions via the `next_mode()` method.

4 Playback Subsystem

The playback subsystem coordinates audio output, queue management, and playback state. It consists of three core components: the audio backend protocol, the music queue, and the high-level player controller.

4.1 Audio Backend Protocol

The `AudioBackend Protocol` defines the interface for all audio playback backends. The current implementation, `VLCAudioBackend`, wraps the `python-vlc` library [2]:

Listing 5. VLCAudioBackend wrapping python-vlc.

```
class VLCAudioBackend:
    def __init__(self) -> None:
        self._instance = vlc.Instance(
            "--quiet", "--no-video-title-show"
        )
        self._player = None

    def _ensure_player(self) -> Any:
        if self._player is None or not hasattr(self._player, "play"):
            self._player = self._instance.media_player_new()
        return self._player

    def play(self, url: str) -> None:
        player = self._ensure_player()
        media = self._instance.media_new(url)
        player.set_media(media)
        player.play()
```

```
18
19      def seek(self, position_ms: int) -> None:
20          if self._player:
21              self._player.set_time(position_ms)
22
23      @property
24      def is_playing(self) -> bool:
25          val = self._player.is_playing()
26          return bool(val) if val is not None else False
```

The backend uses lazy initialization of the `MediaPlayer` instance and caches the `Paused` state enum for fast comparison in the `is_paused` property.

4.2 Music Queue

The `MusicQueue` manages the playback queue with history tracking, repeat modes, and shuffle support:

Architecture Note

Queue invariants: The queue maintains three internal structures:

- `_tracks`: upcoming tracks to play (list)
- `_track_ids`: track IDs for O(1) lookups (set)
- `_history`: played tracks for back navigation (list)
- `_current`: the currently playing track

The `play_next()` method transfers the current track to history and pops the next track from the queue. In `RepeatMode.ALL`, when the queue is exhausted, the history is restored to the queue for a full repeat.

Listing 6. Core queue operations.

```
class MusicQueue:
    def play_next(self) -> Track | None:
        if not self._tracks:
            if self._repeat == RepeatMode.ALL and self._history:
                self._tracks = self._history
                self._history = []
            else:
                return None
        if self._current is not None:
            self._history.append(self._current)
        self._current = self._tracks.pop(0)
        self._track_ids.discard(self._current.id)
        return self._current

    def skip_to(self, track: Track) -> None:
        if track.id not in self._track_ids:
            raise ValueError("Track is not in the queue")
        if self._current is not None:
            self._history.append(self._current)
```

```
        self._tracks.remove(track)
        self._track_ids.discard(track.id)
        self._current = track
```

4.3 Player Controller

The `Player` class sits at the intersection of the audio backend, the queue, and the UI. It handles track transitions, repeat logic, and UI event emission:

Listing 7. Player controller coordinating backend, queue, and UI events.

```
class Player:
    def play(self, track: Track) -> None:
        if not self._queue.contains(track):
            self._queue.add(track)
        self._queue.skip_to(track)
        url = (self._stream_resolver(track)
               if self._stream_resolver else track.url)
        self._backend.play(url)
        self._schedule_ui_update()

    def _update_ui(self) -> None:
        if self._app is None: return
        current = self.current_track
        if current is None: return
        self._app.post_message(
            PlaybackStateChanged(self.is_playing, self.is_paused)
        )
        self._app.post_message(
            PositionUpdated(self.position_ms, self.duration_ms)
        )
        self._app.post_message(QueueUpdated(self.queue))
```

The `on_track_ended()` method implements the repeat logic: `RepeatMode.ONE` restarts the current track, while `RepeatMode.ALL` and `RepeatMode.OFF` advance to the next track in the queue.

5 Catalog Integration

The catalog subsystem provides music discovery and metadata retrieval through integration with YouTube Music and supplementary lyrics APIs.

5.1 YouTube Music Source

The `YTMusicSource` class implements the `CatalogSource Protocol` using the `ytmusicapi` library [6]:

Listing 8. YTMusicSource search and track construction.

```
class YTMusicSource(CatalogSource):
    def search(self, query: str) -> list[Track]:
        client = self._get_client()
        if client is None: return []
        results = client.search(query, filter="songs")
        tracks: list[Track] = []
        seen: set[str] = set()
        for result in results:
            if not result.get("videoId"): continue
            vid = result["videoId"]
            if vid in seen: continue
            seen.add(vid)
            tracks.append(self._make_track(result))
            if len(tracks) >= 12: break
        return tracks

    @staticmethod
    def _make_track(result: dict) -> Track:
        vid = result["videoId"]
        return Track(
            id=zlib.crc32(vid.encode("utf-8")) & 0xFFFFFFFF,
            provider_id=vid,
            title=result.get("title", "Unknown Title"),
            artist=artists[0]["name"] if artists else "Unknown Artist",
            album=(result.get("album") or {}).get("name", "Unknown Album"),
            duration_ms=_duration_to_ms(result.get("duration")),
            image_path=thumbs[-1]["url"] if thumbs else "",
            url=f"https://music.youtube.com/watch?v={vid}",
        )
```

5.2 Lyrics Fetching Pipeline

The lyrics fetching system implements a four-tier fallback pipeline to maximize coverage:

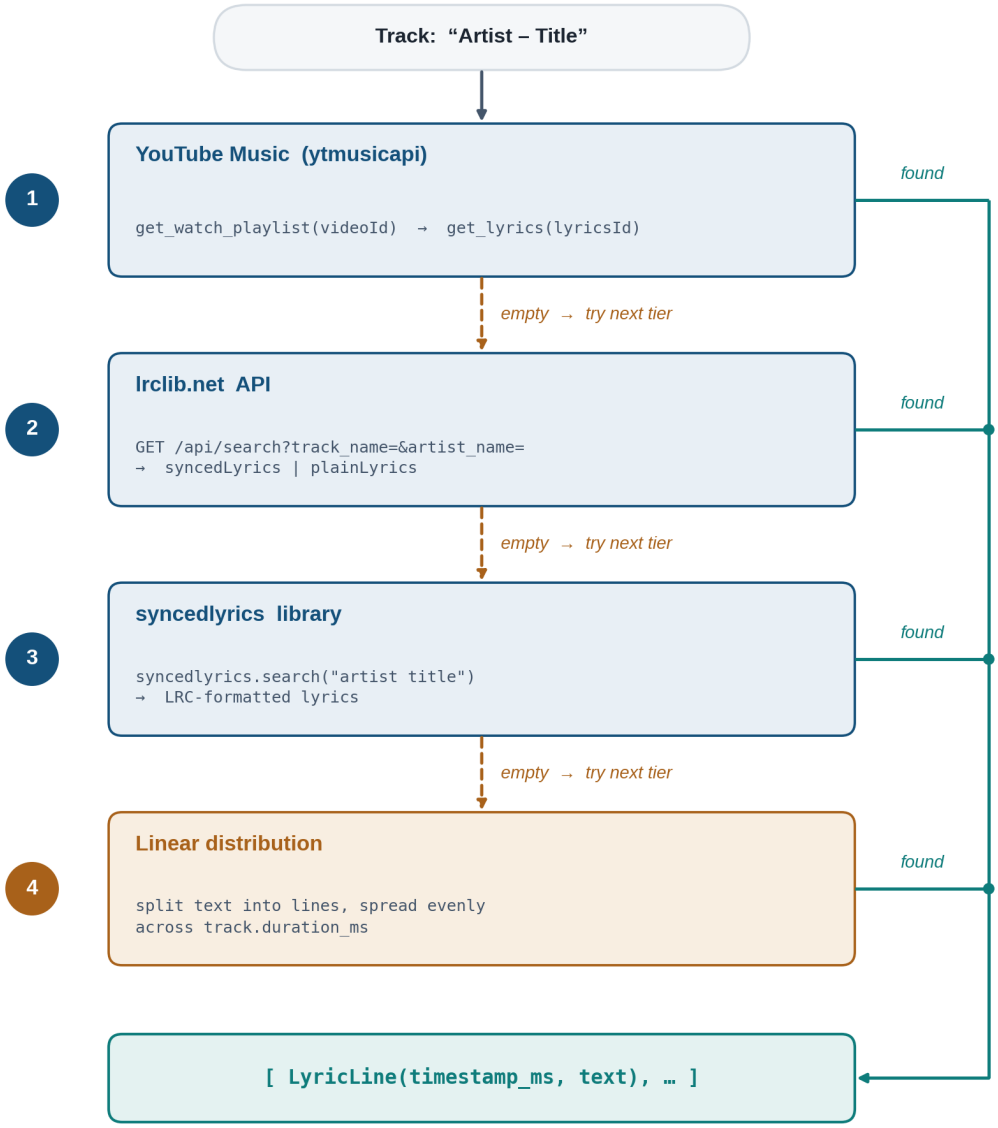


Figure 3. Lyrics fetching pipeline: a four-tier fallback system that maximises coverage by attempting YouTube Music, then lrclib.net, then syncedlyrics, and finally falling back to linear time distribution.

Tier 1: YouTube Music (ytmusicapi) The system first attempts to retrieve lyrics via the YouTube Music watch playlist endpoint. It fetches the watch playlist for the track's video ID, extracts the lyrics bid (identifier), and then retrieves the full lyrics text.

Tier 2: lrclib.net API When YouTube Music lyrics are unavailable, the system queries the lrclib API [8] at `https://lrclib.net/api/search` with the artist and track name. The response (JSON containing synced or plain lyrics) is parsed using the `LyricsParser`.

Tier 3: syncedlyrics Library As a third fallback, the `syncedlyrics` library is invoked to search for synced lyrics using the artist and track name as a query.

Tier 4: Linear Distribution When all API-based sources fail, plain lyrics text is distributed evenly across the track duration using linear interpolation:

$$\text{timestamp}_i = \left\lfloor \frac{i}{N-1} \times \text{duration_ms} \right\rfloor \tag{1}$$

where N is the number of lyric lines and i ranges from 0 to $N-1$.

5.3 LRC Parsing

The `LyricsParser` converts raw `LRC`-formatted text into a list of `LyricLine` objects using regex matching:

Listing 9. LRC timestamp parsing.

```
_LRC_PATTERN = re.compile(
    r"^\[(\d{1,2}):(\d{2})(?:\.(\d{1,3}))?\]\s*(.*)$"
)

@staticmethod
def parse(raw_text: str) -> list[LyricLine]:
    lines: list[LyricLine] = []
    for raw_line in raw_text.splitlines():
        match = _LRC_PATTERN.match(raw_line.strip())
        if match is None: continue
        minutes, seconds, fraction, text = match.groups()
        milliseconds = int((fraction or "0").ljust(3, "0"))
        timestamp_ms = (int(minutes) * 60_000
                        + int(seconds) * 1_000
                        + milliseconds)
        lines.append(LyricLine(timestamp_ms, text))
    return lines
```

The regex handles optional centiseconds (the `.xx` portion), right-padding them with zeros to form a three-digit millisecond value.

5.4 Stream Resolution

Stream URLs are resolved via lazy-loaded `yt-dlp` [7]. The `yt-dlp` module is imported only when needed, and the result is cached in the `TrackCache` keyed by the track's URL:

Listing 10. Lazy yt-dlp stream resolution.

```
def _get_yt_dlp_dl(self) -> Any:
    if self._yt_dlp_dl is not None:
        return self._yt_dlp_dl
    try:
        yt_dlp = import_module("yt_dlp")
        options = {
```

```
            "format": "bestaudio/best",
            "quiet": True,
            "noplaylist": True,
            "cachedir": True,
        }
        self._yt_dlp_dl = yt_dlp.YoutubeDL(options)
    except ImportError:
        self._yt_dlp_dl = None
    return self._yt_dlp_dl

def get_track_stream_url(self, track: Track) -> str:
    dl = self._get_yt_dlp_dl()
    if dl is None:
        return track.url
    info = dl.extract_info(track.url, download=False)
    return info.get("url", track.url)
```

6 User Interface

The UI is built on the `Textual` framework, which provides a reactive component model with CSS-like styling, keyboard bindings, and built-in support for asynchronous workers.

6.1 Application Structure

The `MusicTUI`(App) class manages the application lifecycle, service initialization, screen navigation, and asset preloading:

Listing 11. Main application class wiring all services together.

```
class MusicTUI(App):
    CSS_PATH = "style.tcss"
    BINDINGS = [
        Binding("q", "quit", "Quit"),
        Binding("h", "go_home", "Home"),
        Binding("s", "go_search", "Search"),
        Binding("l", "go_library", "Library"),
        Binding("f", "toggle_fullscreen", "Fullscreen"),
    ]

    def on_mount(self) -> None:
        self._event_bus = EventBus(self)
        self._catalog_source = YTMusicSource()
        self._store = Database("store.db")
        self._track_cache = TrackCache(max_size=50)
        self._playlist_store = PlaylistStore(
            self._store, self._catalog_source
        )
        self._catalog_service = CatalogService(
            self._catalog_source, self._track_cache
        )
        self._library_service = LibraryService(self._playlist_store)
        self._player = Player(
            backend=VLCAudioBackend(),
```

```
            queue=MusicQueue(),
            stream_resolver=lambda t:
                self._catalog_service.get_stream(t),
        )
        self._player.set_textual_app(self)
        self._playback_service = PlaybackService(
            self._player, self._event_bus
        )
        self.push_screen(HomeScreen(
            self._catalog_service,
            self._playback_service,
            self._library_service
        ))
```

6.2 Screen Architecture

`MusicTUI` defines six screen classes, each responsible for a specific view of the application:

Table 1. Screen overview table describing each screen's purpose and key features.

Screen	Description
`HomeScreen`	Recommended tracks with infinite scroll, search bar, next-track card, and now-playing bar
`SearchScreen`	Query input with search results in a track list, supporting like and add-to-playlist actions
`LibraryScreen`	Two-panel layout: playlist table (left) and song list (right) with CRUD operations
`PlaylistScreen`	Single playlist view with track playback, navigation, and song removal
`FullscreenScreen`	Large album-art overlay filling the terminal, with track title/artist caption
`LyricsScreen`	Synced lyrics overlay with auto-highlighting, loaded via background worker

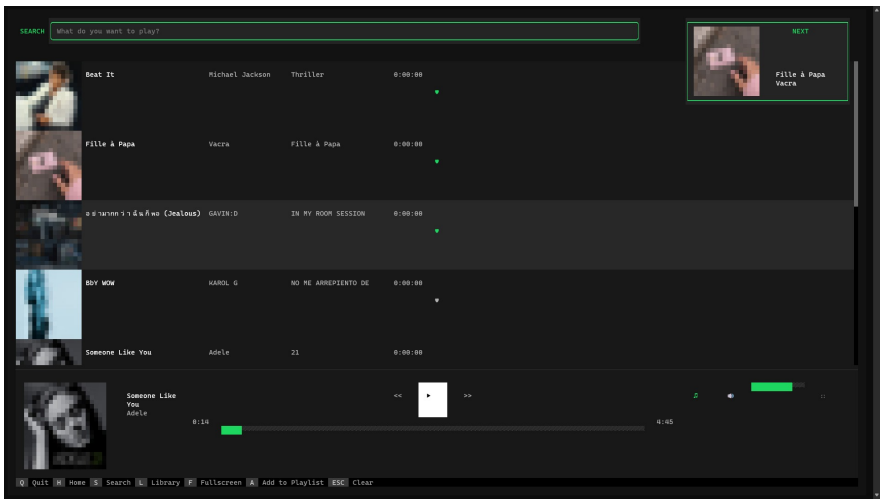


Figure 4. The home screen layout showing the two-column grid with sidebar navigation, recommended tracks list, next-track preview, and the persistent now-playing bar at the bottom.

6.3 Widget Hierarchy

The application uses 22 reusable widget classes. Figure 5 shows the main groupings.

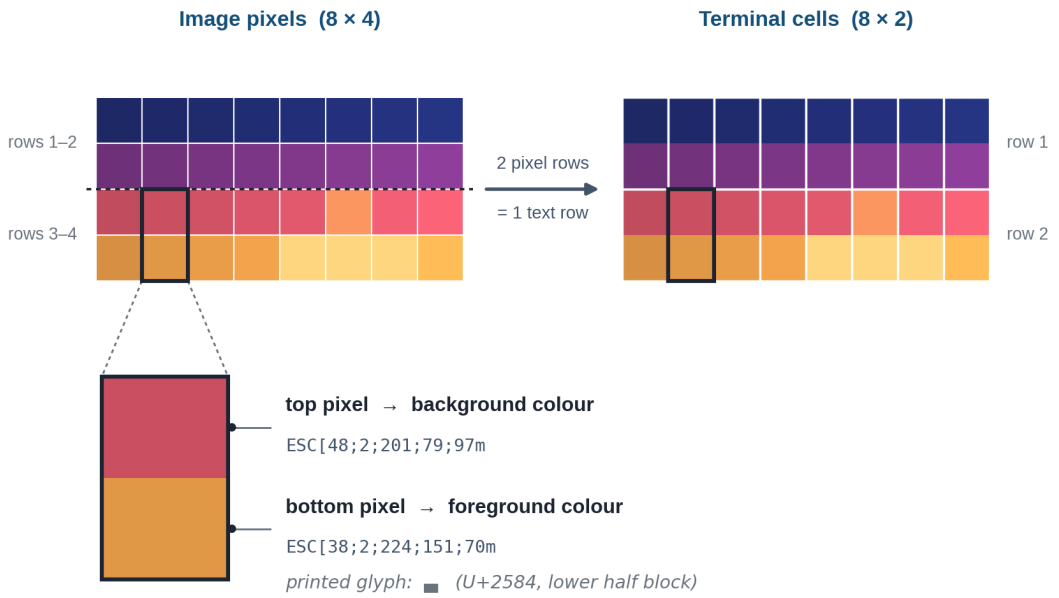


Figure 5. ANSI half-block colour rendering: each terminal character combines two pixel rows – the top row as background colour and the bottom row as foreground colour – producing a dense, full-colour image.

6.3.1 Now-Playing Bar

The `NowPlayingBar` is the most complex widget, composed of three sub-sections:

- **Track info** (`TrackInfo`): Album art (`AlbumArt`) + title/artist labels. Reacts to `EventBus` messages to update displayed information.
- **Playback center** (`PlaybackCenter`): Transport controls (`PlayerControls`: prev, play/pause, next) + seek bar (`ProgressRow`: current time + `TrackProgress` slider + duration).
- **Utilities** (`PlayerUtilities`): Lyrics toggle button + `VolumeControl`(mute button + volume slider) + fullscreen toggle.

The bar uses `Textual`'s reactive properties to respond to state changes.

6.3.2 Album Art Rendering

The `AlbumArt` widget renders album artwork as `ANSI` half-block colour text. It uses a background worker to perform the computationally intensive downscaling operation.

Listing 12. AlbumArt widget rendering.

```
class AlbumArt(Static):
    def _render_artwork(self) -> None:
```

```
        self._render_generation += 1
        cols = self._art_cols or round(
            self.size.height * CELL_ASPECT
        )
        self.run_worker(
            partial(downscale, self._source,
                    cols=cols, cell_aspect=CELL_ASPECT),
            name="album-art-render", thread=True,
        )

    def on_worker_state_changed(self, event):
        if event.state == WorkerState.SUCCESS:
            self.update(Text.from_ansi(event.worker.result))
```

6.4 ANSI Half-Block Colour Rendering

The `downscaler` module converts any image (local file or URL) to `ANSI` half-block colour text using `PIL` and `NumPy` [5, 4]. The algorithm:

1. Load the image using `PIL`(Pillow).
2. Resize to W columns by $H = 2 \times$ rows pixels using BICUBIC interpolation.
3. For each pair of pixel rows $(2i, 2i+1)$, combine the top pixel (background colour) and bottom pixel (foreground colour) into a single `ANSI` half-block character.
4. Each character is encoded as an escape sequence: `\e[48;2;R;G;Bm\e[38;2;r;g;bm\2584`.

Listing 13. The core downscaling algorithm.

```
_ANSI_FMT = "\x1b[48;2;{};{};{}m\x1b[38;2;{};{};{}m\u2584"

def downscale(image_path: str, cols: int = 24,
              cell_aspect: float = 2.1) -> str:
    text_rows = max(1, round(cols / cell_aspect))
    W, H = cols, text_rows * 2
    image = Image.open(BytesIO(_read_image(image_path)))
    image = image.convert("RGB")
    image = image.resize((W, H), Image.BICUBIC)
    arr = np.asarray(image)
    lines = []
    for ty in range(0, H, 2):
        top = arr[ty].tolist()
        bot = arr[ty + 1].tolist()
        line_parts = [
            _ANSI_FMT.format(a[0], a[1], a[2], b[0], b[1], b[2])
            for a, b in zip(top, bot)
        ]
        lines.append("".join(line_parts))
    return "\n".join(lines)
```

The algorithm is cached via `@lru_cache(maxsize=64)`, avoiding redundant computation.

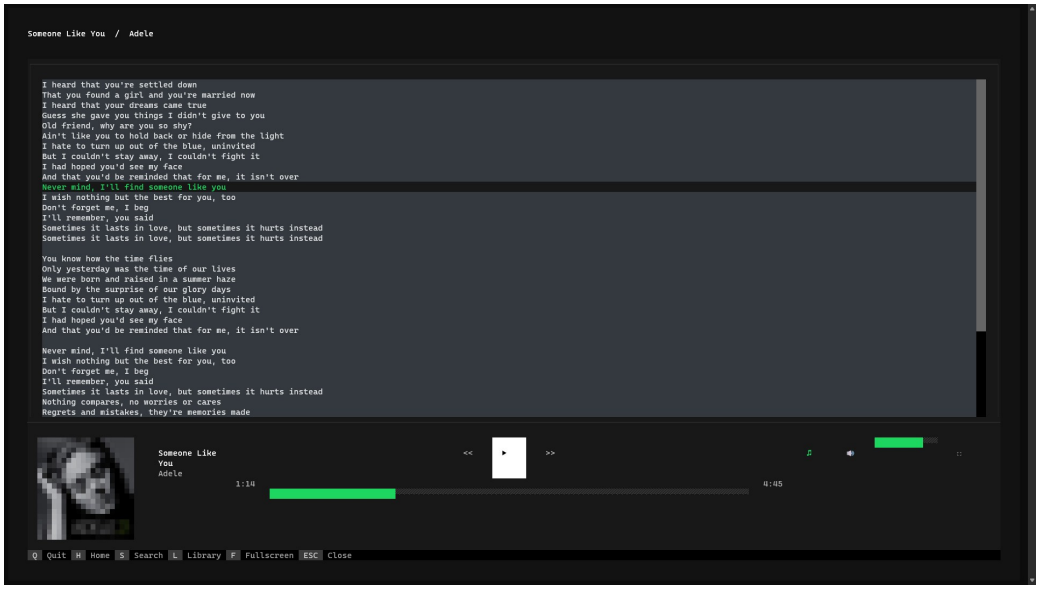


Figure 6. The lyrics overlay screen showing synchronized lyrics with the currently active line highlighted in green, loaded via a background worker and highlighted every 250ms using binary search.

6.5 Event-Driven Communication

The `EventBus` decouples the player from UI widgets. Instead of widgets polling the player for state, the player posts messages to the `EventBus`, and widgets subscribe to the relevant message types:

Listing 14. Event types and EventBus.

```
class EventBus:
    def __init__(self, app: App) -> None:
        self._app = app

    def emit(self, message: Message) -> None:
        self._app.post_message(message)

    def emit_playback_started(self, track: Track) -> None:
        self.emit(PlaybackStarted(track))

    def emit_position_updated(
        self, position_ms: int = 0, duration_ms: int = 0
    ) -> None:
        self.emit(PositionUpdated(position_ms, duration_ms))
```

Message types include: `PlaybackStarted`, `PlaybackEnded`, `PlaybackStateChanged`, `PositionUpdated`, `QueueUpdated`, `LikedSongsChanged`, and `PlaylistChanged`.

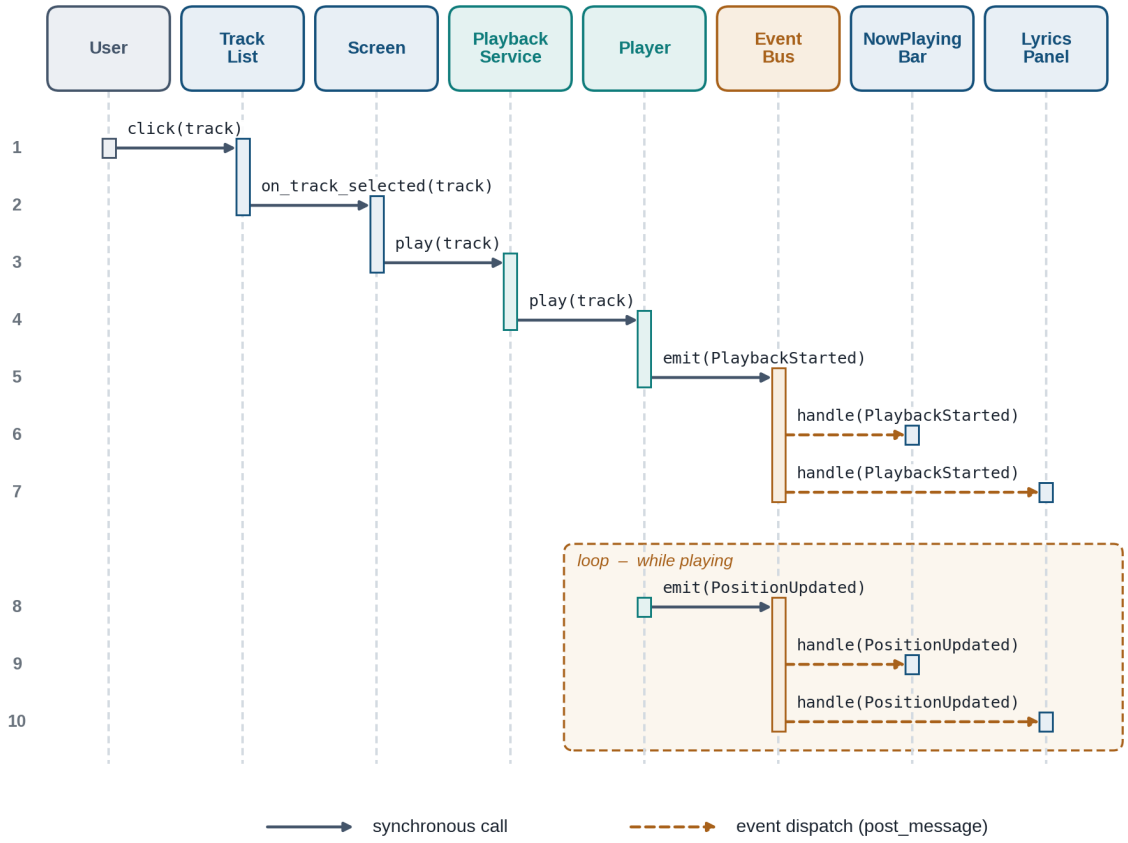


Figure 7. Sequence diagram showing the event flow when a user clicks a track: the click propagates through the track list widget to the screen, which calls the playback service, which notifies the player, which posts messages to the event bus, which are consumed by all subscribed widgets.

6.6 Library Screen

The `LibraryScreen` provides a two-panel layout for playlist management. The left panel is a `DataTable` listing all playlists, while the right panel displays the songs in the selected playlist:

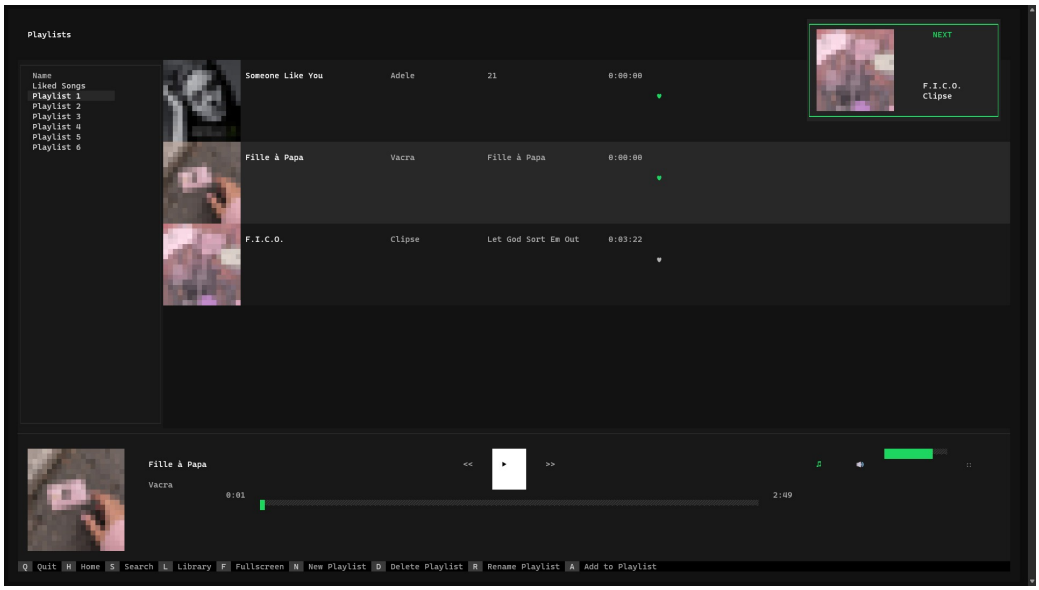


Figure 8. The library screen with the two-panel layout: playlist table on the left and song list on the right. Supports playlist CRUD operations (create, rename, delete) via modal dialogs.

6.7 Modal Dialogs

Three modal screen classes handle user input: `NewPlaylistScreen`, `RenamePlaylistScreen`, and `PlaylistPickerScreen`. All modals support `Escape`-to-cancel and `Enter`-to-submit shortcuts.

6.7.1 Fullscreen Screen

The `FullscreenScreen` provides a full-terminal overlay showing a large `ANSI` half-block colour rendering of the current track's album art:

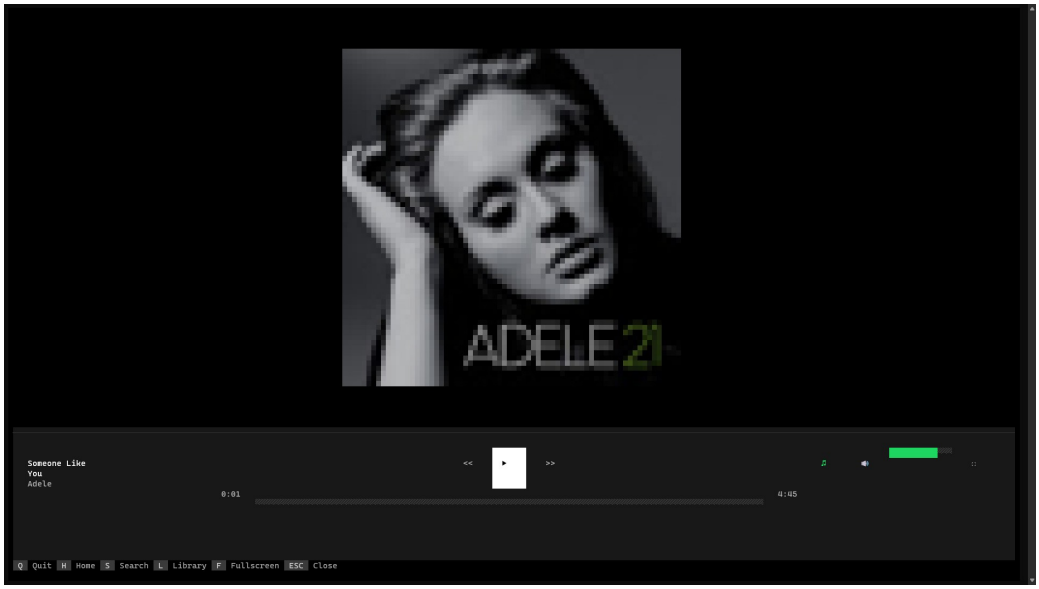


Figure 9. The fullscreen overlay displaying a large `ANSI` half-block colour rendering of the current track's album art, filling the terminal window.

7 Storage Layer

The storage layer provides persistent data management using `SQLite` [3], a lightweight, serverless, zero-configuration SQL database engine.

7.1 Database Wrapper

The `Database` class wraps an `SQLite` connection with context manager support and automatic schema management:

Listing 15. Database wrapper with migration.

```
class Database:
    def __init__(self, db_path: str) -> None:
        self._db_path = Path(db_path)
        self._connect()

    def _connect(self) -> None:
        self._conn = sqlite3.connect(str(self._db_path))
        self._conn.row_factory = sqlite3.Row
        rows = self._conn.execute(
            "PRAGMA table_info(liked_songs)"
        ).fetchall()
        if "track_id" in [r["name"] for r in rows]:
            self._conn.execute("DROP TABLE IF EXISTS liked_songs")
        self._conn.execute(
            "CREATE TABLE IF NOT EXISTS liked_songs "
            "(provider_id TEXT PRIMARY KEY)"
        )
        self._conn.execute(
            "CREATE TABLE IF NOT EXISTS tracks "
            "(id INTEGER PRIMARY KEY AUTOINCREMENT, "
```

```
            "provider_id TEXT UNIQUE, title TEXT, artist TEXT, "
            "album TEXT, duration_ms INTEGER, "
            "image_path TEXT, lyrics_path TEXT, url TEXT)"
        )
        self._conn.execute(
            "CREATE TABLE IF NOT EXISTS playlists "
            "(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL)"
        )
        self._conn.execute(
            "CREATE TABLE IF NOT EXISTS playlist_songs "
            "(playlist_id INTEGER NOT NULL, track_id INTEGER NOT NULL, "
            "FOREIGN KEY(playlist_id) REFERENCES playlists(id), "
            "FOREIGN KEY(track_id) REFERENCES tracks(id), "
            "PRIMARY KEY(playlist_id, track_id))"
        )
        self._conn.commit()

    def __enter__(self) -> Self: return self
    def __exit__(self, *args: object) -> None: self.close()
```

The schema includes four tables: `tracks`, `playlists`, `playlist_songs` (junction table with foreign keys), and `liked_songs`.

7.2 Playlist and Liked-Songs Stores

The `PlaylistStore` class manages playlist and song persistence, resolving stored track IDs back to `Track` objects. The `LikedSongStore` manages the liked-songs collection using `INSERT OR IGNORE` to prevent duplicates.

8 Testing

`MusicTUI` includes a comprehensive test suite organized in parallel to the production code structure, using `pytest` [9].

8.1 Test Structure

```
tests/
├── catalog/
│   ├── fake_catalog_source.py      # In-memory mock for CatalogSource
│   ├── test_fake_catalog_source.py # Search functionality tests
│   ├── test_lyrics_parser.py       # LRC parsing edge cases
│   ├── test_lyrics_provider.py     # Lyrics fetch tests
│   └── test_track_cache.py         # LRU cache + TrackCache tests
├── models/
│   ├── test_play_state.py          # Immutable state tests
│   ├── test_playlist.py            # CRUD + navigation tests
│   ├── test_repeat_mode.py         # Cycle tests
│   └── test_track.py               # Construction + replace tests
├── playback/
│   ├── fake_backend.py             # In-memory mock for AudioBackend
│   ├── test_music_queue.py         # Queue operation tests
│   └── test_player.py              # Player operation tests
```

```
└── store/
    ├── test_liked_songs_store.py   # SQLite in-memory tests
    └── test_playlist_store.py      # Playlist persistence tests
```

8.2 Mock Implementations

Two key mock classes enable unit testing without external dependencies:

Listing 16. FakeBackend mock for testing.

```python
class FakeBackend(AudioBackend):
    def __init__(self):
        self._playing = False
        self._paused = False
        self._volume = 75
        self._position_ms = 0

    def play(self, url: str) -> None:
        self._playing = True
        self._paused = False

    def pause(self) -> None: self._paused = True
    def resume(self) -> None: self._paused = False
    def stop(self) -> None: self._playing = False

    def seek(self, position_ms: int) -> None:
        self._position_ms = position_ms

    def get_position(self) -> int: return self._position_ms
    def get_duration(self) -> int: return 180000
```

8.3 Test Categories

Model Tests (36 tests) Model tests verify frozen dataclass construction, the `replace()` functional update pattern, immutability guarantees, hash/equality semantics, and normalization edge cases. The `Track` model is tested for all default values, field normalization (whitespace-only fields fall back to defaults), negative duration clamping, CRC32 ID generation, and hash consistency for use in sets and dictionaries. The `Playlist` model is tested for O(1) membership via set-backed lookup, cursor position tracking on add/remove, and the `next_song` / `previous_song` navigation methods.

Listing 17. Immutability test for frozen dataclass.

```python
class TestTrack:
    def test_immutability(self):
        t = Track(id=1, title="Test")
        try:
            t.title = "Changed"
            assert False, "Should not mutate frozen dataclass"
        except Exception:
            pass

```

```
    def test_hash_and_equality(self):
        t1 = Track(id=1, title="Test")
        t2 = Track(id=1, title="Different")
        assert t1 == t2  # Same id => equal
        assert hash(t1) == hash(t2)
        s = {t1}
        assert t2 in s  # Works in sets
```

Cache Tests (16 tests) The `TrackCache` and its underlying `_LRUCache` generic LRU cache are tested for eviction policy, LRU promotion (accessing a key moves it to the most-recently-used position), key normalization for lyrics (whitespace stripping and case-insensitive matching), and independent sub-caches for tracks, lyrics, and stream URLs.

Listing 18. LRU promotion test.

```
class TestLRUCache:
    def test_lru_promotion(self):
        cache = _LRUCache(max_size=3)
        cache.put("a", 1)
        cache.put("b", 2)
        cache.put("c", 3)
        cache.get("a")  # promotes "a" to MRU
        cache.put("d", 4)  # evicts "b" (was LRU)
        assert cache.get("a") == 1
        assert cache.get("b") is None  # evicted
```

Queue and Player Tests (41 tests) Queue tests verify FIFO ordering, history management, repeat mode cycling, `skip_to()` validation, duplicate prevention, shuffle, and the `replace_upcoming()` operation. Player tests use `FakeBackend` to verify play/pause toggling, seek, volume control, position tracking, stream resolver integration, and the `on_track_ended()` repeat handling logic.

Catalog Mock Tests (8 tests) The `FakeCatalogSource` mock is tested for case-insensitive search across title, artist, and album fields, empty query handling, and multiple match returns.

Storage Tests (24 tests) Storage tests use in-memory `SQLite` databases (`Database(":memory:")`) to verify CRUD operations, foreign key constraints, provider-based track resolution (ensuring `zlib.crc32` in-memory IDs resolve correctly to the database primary key), and the liked-songs store ordering.

8.4 Running Tests

Listing 19. Running the test suite.

```
$ cd music/
$ python -m pytest tests/ -v
======================== 167 passed in 0.16s ========================
$ python -m pytest tests/ -v --cov=music --cov-report=term-missing
```

The test suite covers all production modules in parallel: catalog (parsing, caching, search mocks), models (frozen dataclasses, enum cycling), playback (queue operations, player controls), and store (SQLite persistence). Tests run in under one second, enabling rapid feedback during development.

9 Getting Started

9.1 Prerequisites

- `Python` 3.12 or later
- `VLC` media library (system package)
- A terminal emulator with 24-bit colour support (e.g., Windows Terminal, iTerm2, GNOME Terminal)

9.2 Installation

Listing 20. Installation instructions.

```
$ git clone https://github.com/username/musictui.git
$ cd music
$ python -m venv .venv
$ source .venv/bin/activate  # On Windows: .venv\Scripts\activate
$ pip install -e ".[dev]"
$ python -m pytest tests/ -v
```

9.3 Running the Application

Listing 21. Launching the application.

```
$ python main.py
```

9.4 Keyboard Shortcuts

Table 2. Keyboard shortcuts available in `MusicTUI`.

Key	Mode	Action
`h`	Always	Navigate to Home screen
`s`	Always	Navigate to Search screen
`l`	Always	Navigate to Library screen
`f`	Always	Toggle fullscreen album art
`q`	Always	Quit application
`a`	Home/Search	Add currently playing track to playlist
`n`	Library	Create new playlist
`d`	Library/Playlist	Delete playlist / Remove song
`r`	Library	Rename selected playlist
`Escape`	Lyrics/Fullscreen	Close overlay screen
`Enter`	Search bar	Submit search query
`Escape`	Search input	Clear search input

10 Future Work

Several areas have been identified for future enhancement of `MusicTUI`:

10.1 Additional Music Providers

The `CatalogSource Protocol` is designed to support multiple backends. Planned implementations include: local file system provider with `mutagen` metadata extraction, Spotify integration via `spotipy`, and Deezer integration via `deezer-python`.

10.2 Enhanced Lyrics Features

- **Karaoke-style word-level highlighting:** Current lyrics display line-level highlighting. Word-level timing data from lrclib's `syncedLyricsPlain` field could enable per-word highlighting.
- **Offline lyrics caching:** Cache fetched lyrics to disk for offline playback.
- **Multi-language lyrics:** Display lyrics in multiple languages simultaneously when available.

10.3 Audio Enhancements

- **Equalizer:** Real-time audio equalization using `VLC`'s built-in equalizer presets.
- **Crossfade:** Smooth transitions between tracks.
- **Audio visualization:** Real-time waveform or frequency spectrum visualization using `NumPy`'s FFT capabilities.

- **Multi-backend support:** Add `pygame.mixer` or `portaudio` backends for systems where `VLC` is unavailable.

10.4 UI Improvements

- **Theme system:** Allow users to define custom colour themes via a `themes/` directory of `.tcss` files.
- **Responsive layouts:** Adapt the grid layout for small terminals (below 80 columns) by hiding the sidebar.
- **Mouse support:** Full mouse interaction for track selection, slider dragging, and button clicking across all screens.
- **Recently played screen:** Display a history of recently played tracks with the ability to re-play.

10.5 Data and Analytics

- **Listening statistics:** Track total listening time, most played artists, and most played tracks.
- **Export/import:** Export playlists as `.m3u` or `.xspf` files; import from `.m3u`, `.xspf`, or Spotify CSV exports.

11 Conclusion

This paper has presented `MusicTUI`, a terminal-based music player that demonstrates how modern Python frameworks can produce rich, interactive applications within the terminal environment. The project showcases several key software engineering practices:

- **Protocol-based architecture** provides a clean separation of concerns while maintaining flexibility for future extensions.
- **Event-driven communication** through the `EventBus` eliminates tight coupling between components and enables independent evolution of the playback engine and UI layers.
- **Immutable domain models** using frozen dataclasses with slots ensure thread safety and memory efficiency in a multi-threaded environment.
- **ANSI half-block colour art** provides a unique visual experience that leverages the terminal's capabilities in a way that desktop GUI applications cannot replicate.
- **Multi-tier lyrics fetching** ensures maximum coverage for synchronized lyrics through a systematic fallback chain.
- **Comprehensive testing** with 167 unit tests across 10 test modules, using in-memory mocks and SQLite in-memory databases, provides confidence in the correctness of the application logic without requiring external dependencies.

The `MusicTUI` project demonstrates that terminal-based applications can compete with desktop GUI applications in terms of features and user experience, while maintaining the

advantages of terminal applications: lightweight resource usage, SSH accessibility, and keyboard-driven efficiency.

References

[1] Textual Framework. https://textual.textualize.io/ A library for building terminal user interfaces in Python.

[2] VideoLAN. https://www.videolan.org/vlc/ VLC media library – a free and open-source cross-platform multimedia framework and player.

[3] SQLite Consortium. https://www.sqlite.org/ SQLite – A C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine.

[4] Pillow (PIL Fork). https://pillow.readthedocs.io/ The Python Imaging Library – adds image processing capabilities to your Python interpreter.

[5] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., & Oliphant, T. E. Array programming with numpy. *Nature*, 585(7825), 357–362, 2020.

[6] nathom. https://github.com/nathom/ytmusicapi A Python API for interacting with YouTube Music.

[7] yt-dlp contributors. https://github.com/yt-dlp/yt-dlp A youtube-dl fork with additional features and fixes.

[8] lrclib. https://lrclib.net/ A community-driven API for obtaining synced and plain lyrics.

[9] pytest contributors. https://docs.pytest.org/ A framework for writing simple and scalable tests in Python.

[10] Pierre Raybaut. https://github.com/nickolag/python-vlc Python bindings for the VLC media library.

[11] Python Software Foundation. https://www.python.org/ Python 3.12 – What's New in Python 3.12.